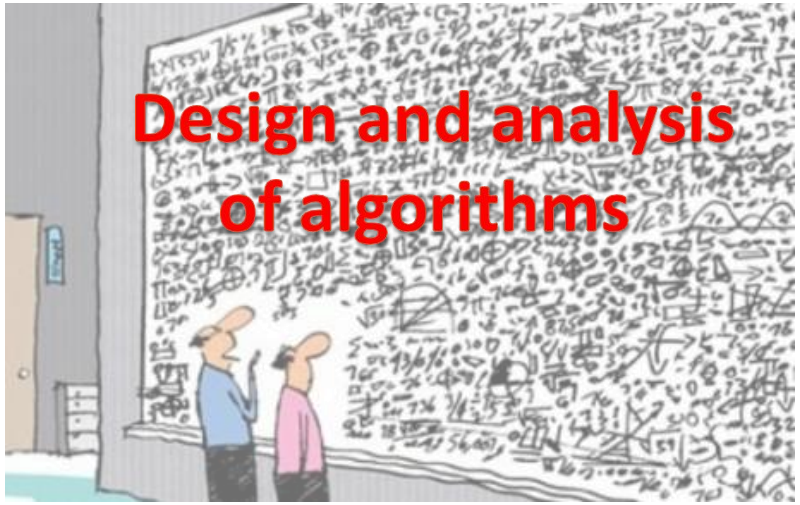
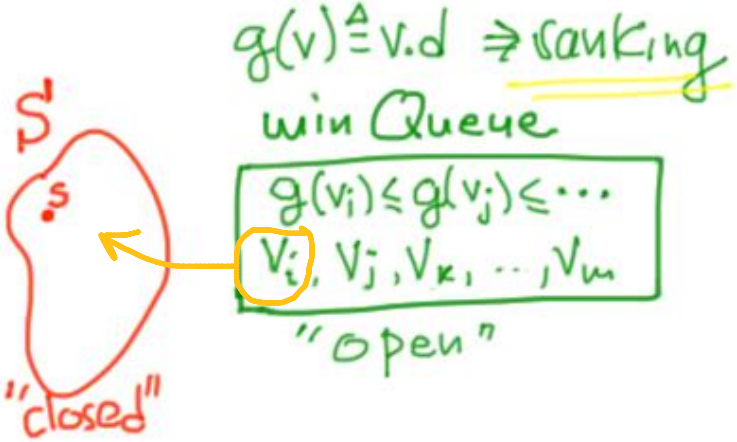


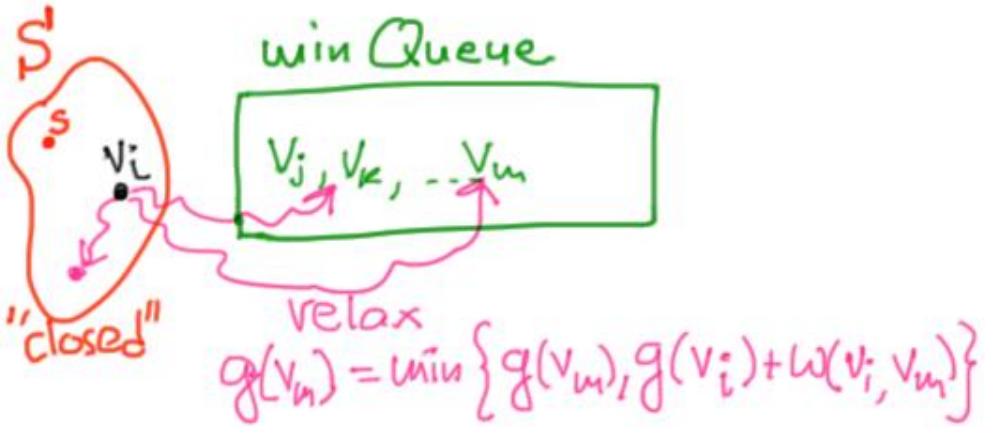
- **A***

Objective

- Understand A*
- https://www.youtube.com/watch?v=cSxnOm5aceA
- https://brilliant.org/wiki/a-star-search/
- more details: http://theory.stanford.edu/~amitp/GameProgramming/ImplementationNotes.html
- Do your own experiment in class: https://qiao.github.io/PathFinding.js/visual/

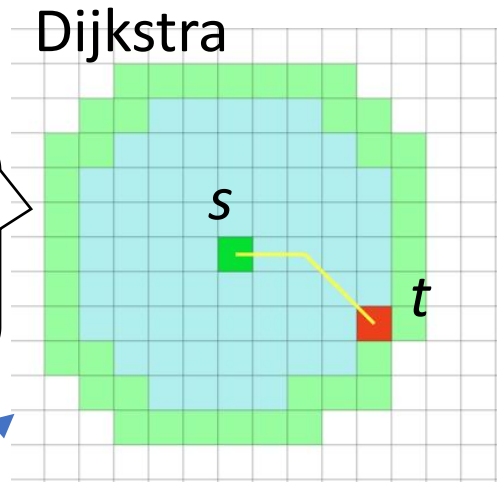
Dijkstra's algorithm (reminder)

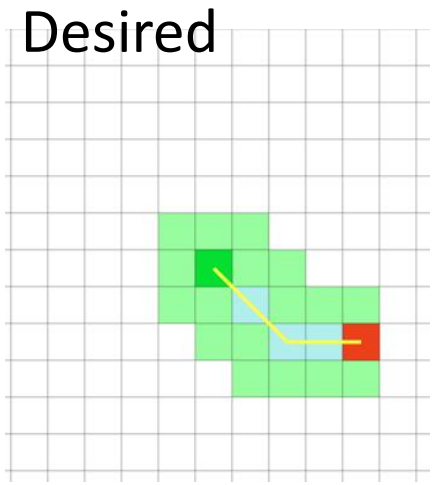




Goal-directed search (A* algorithm)

- Our searches are on the plane, Euclidean (or Manhattan) distance.
- Dijkstra: explore nodes in a "circular" way around *s*.
- Suppose we have some information about the location of the destination, can we add it to the search to find faster the shortest path?





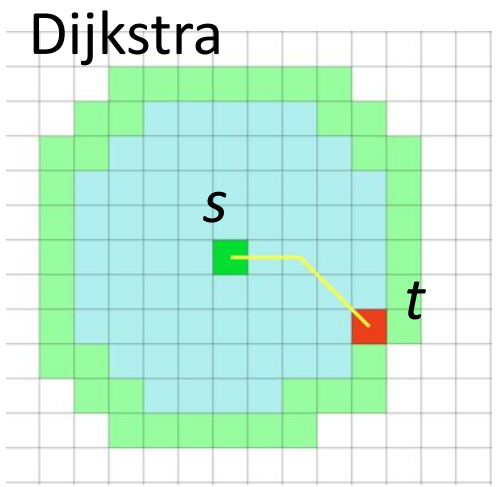
dark green: s
red: t
light green: in set Q,
i.e., partially explored,
blue: already explored.

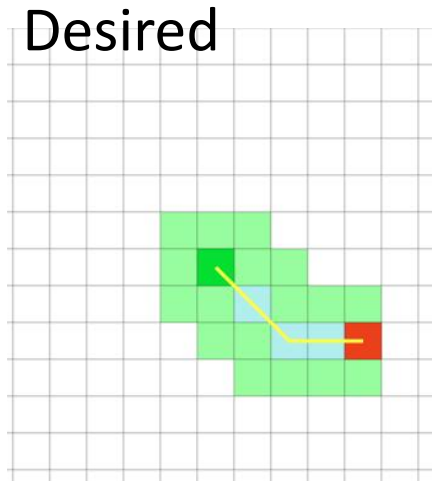
Can we find the shortest path to t by exploring fewer nodes?
Change the priority of exploring new nodes?

how do I know I found a shortest path?

- Dijkstra: explore nodes with priority $g(v)$ = lowest "past" cost, i.e., cost to get to v. How to change?

Goal-directed search (A* algorithm)





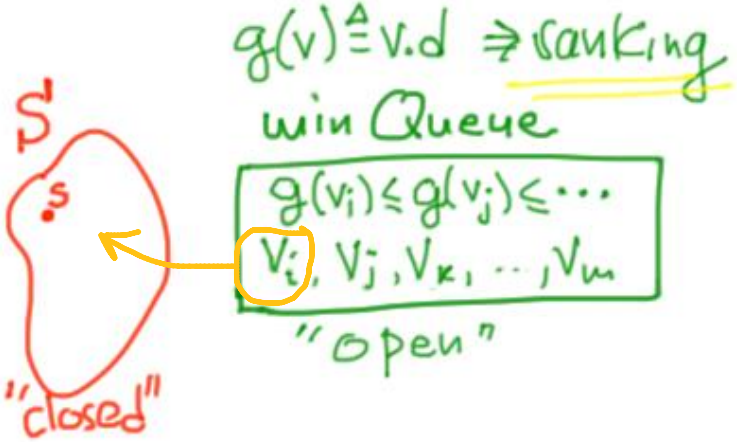
Why bother if we know the destination in grid type graphs?

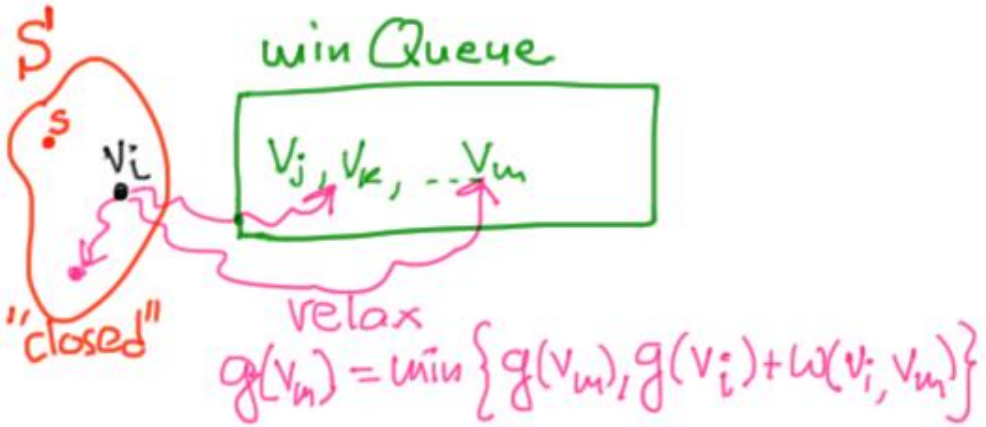
- When considering node *v* from *Q*:
- Dijkstra: priority = $g(v)$: cost to reach v = **past cost**.
- A^*: priority is $f(v) = g(v) + h(v)$.

$h(v)$: heuristic to estimate of **future cost** *from* v to t.

Note for **arbitrary** graphs: If we could guess correctly $\delta(v,t)$, and set $h(v) = \delta(v,t)$, then A* would move along the shortest path. Because $f(v) = g(v) + h(v) =$ shortest path from *s* to *v* + shortest path from *v* to *t*, ⇒ is minimized if v on shortest path from s to t.

Dijkstra's algorithm (reminder)





A*

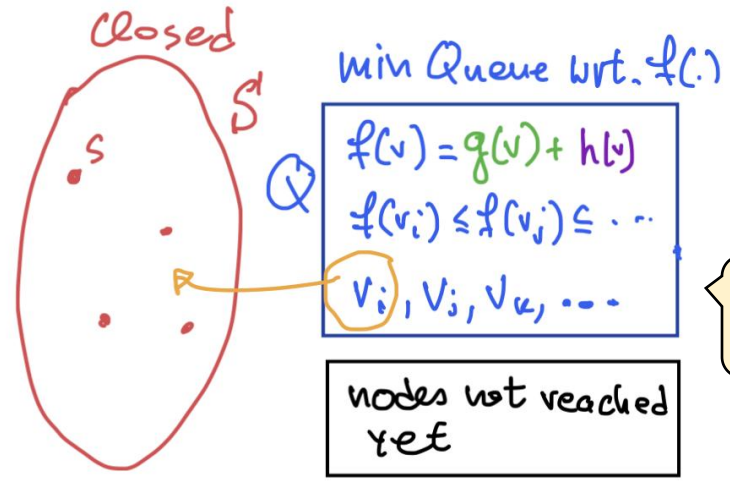
```
OPEN = priority queue containing START
CLOSED = empty set
while lowest rank in OPEN is not the GOAL:
  current = remove lowest rank item from OPEN
  add current to CLOSED
  for neighbors of current:
    cost = g(current) + movementcost(current, neighbor)
(a) if neighbor in OPEN and cost less than g(neighbor):
      remove neighbor from OPEN, because new path is better
(b) if neighbor in CLOSED and cost less than g(neighbor): **
      remove neighbor from CLOSED
(c) if neighbor not in OPEN and neighbor not in CLOSED:
      set g(neighbor) to cost
      add neighbor to OPEN
      set priority queue rank to g(neighbor) + h(neighbor)
      set neighbor's parent to current

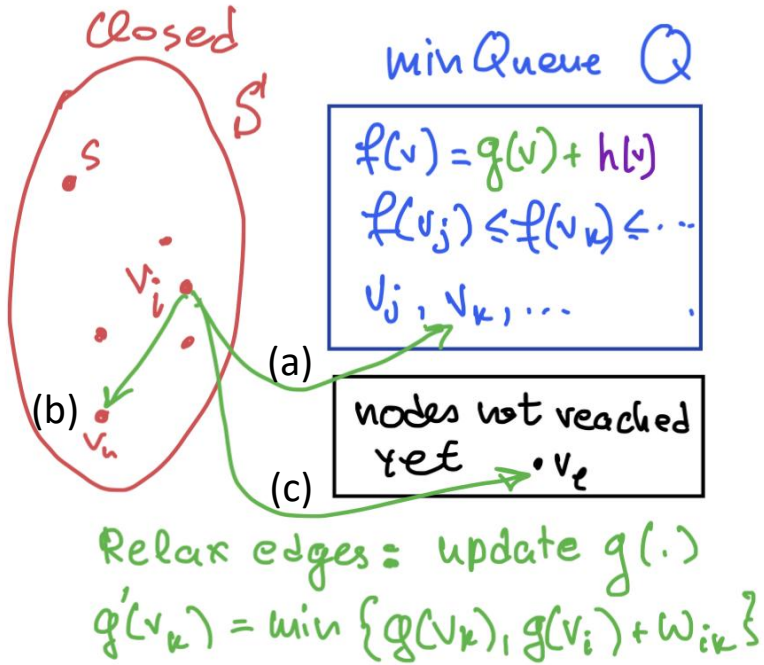
reconstruct reverse path from goal to start
by following parent pointers
```

$f(\text{neighbor})$

(**) This should never happen if you have an consistent admissible heuristic. However in games we often have inadmissible heuristics.

Bring into closed set according to $f(.)$, relax edges according to $g(.)$, i.e., update g.

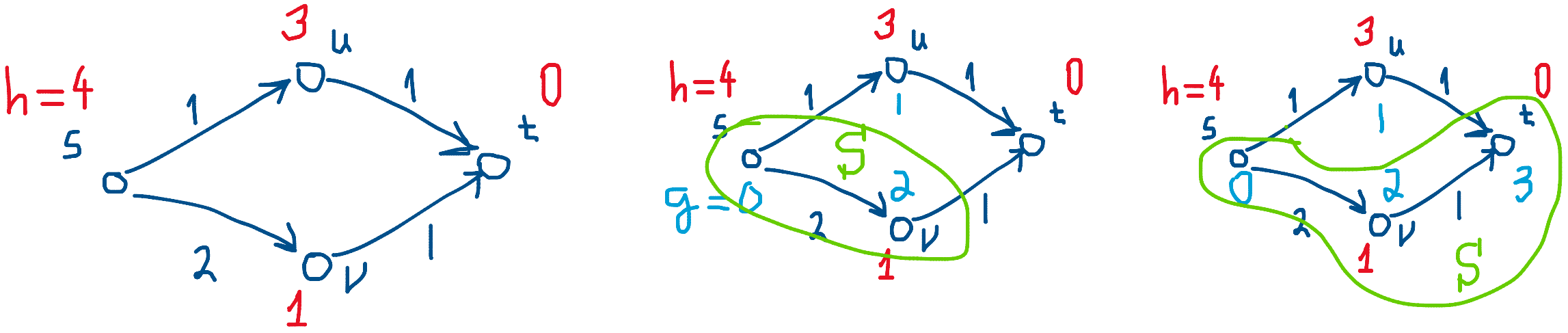




when we bring a node *v* to the closed set, $g(v)$ may not be $\delta(s, v)$.

Exercise

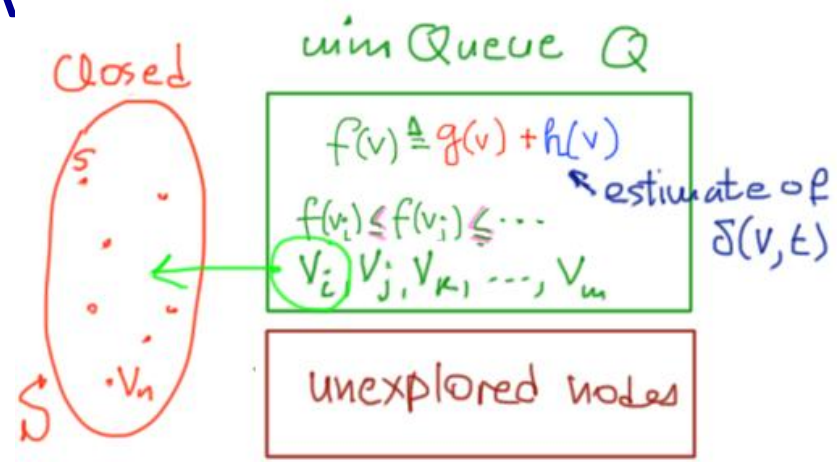
- Run A* on the following graph and check that it will terminate with the wrong shortest path to t.

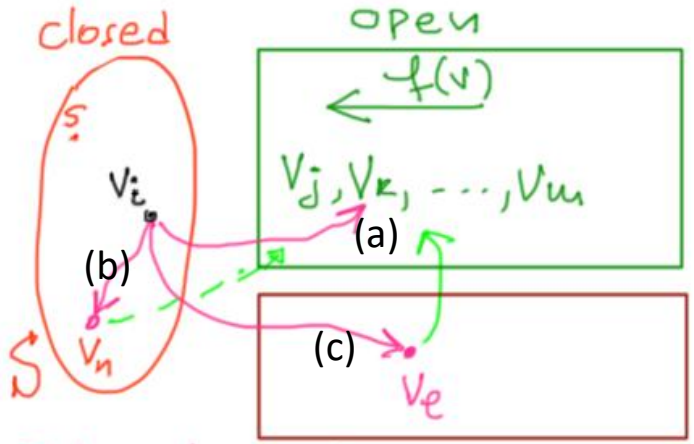


- Why? because h not "admissible": must have $h(v) \leq \delta(v, t)$.
- Other issues? nodes may go in and out of S: no guarantee of termination time. Need h to be "consistent". Consistency implies admissibility.

A*

g: distance cost, h: future cost estimate





Relax edges: if $g' < g$ then update g, parent node. Move node to open set

--> : no need if $h()$ is "consistent"

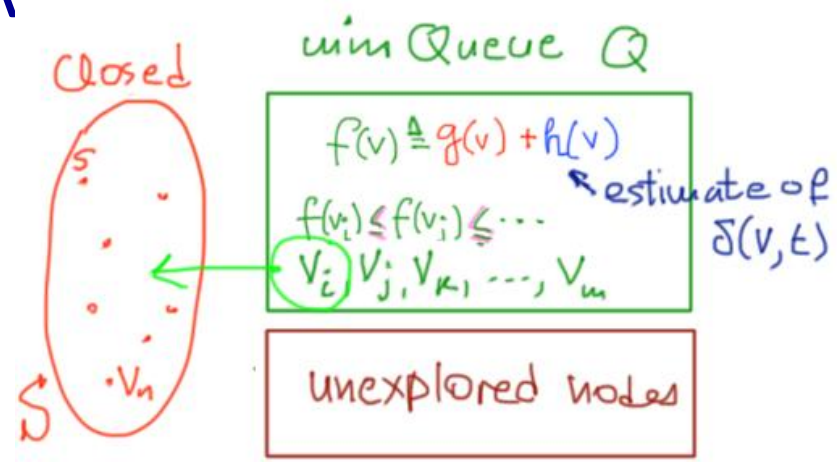
To get any guarantees on A* we need to restrict $h(.)$.

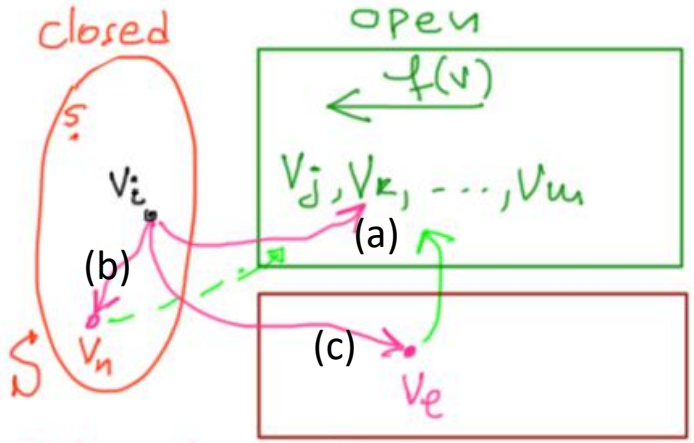
Admissible $h(.)$: A* always terminates successfully by finding the shortest path.

- Iff $h(v) \leq \delta(v, goal)$.

$h(v)$ never overestimates the actual cost to get to the goal.

A*





Relax edges: if $g' < g$ then update g, parent node. Move node to open set
--> : no need if $h()$ is "consistent"

To get any guarantees on A* we need to restrict $h(.)$.

Admissible $h(.)$: A* always terminates successfully by finding the shortest path.

- Iff $h(v) \leq \delta(v, goal)$.

Consistent $h(.)$: A* always terminates successfully and never moves nodes from the closed back to the open set.

- Iff $w'(u,v) = w(u,v) + h(v) - h(u) \geq 0, \ \forall (u,v)$.

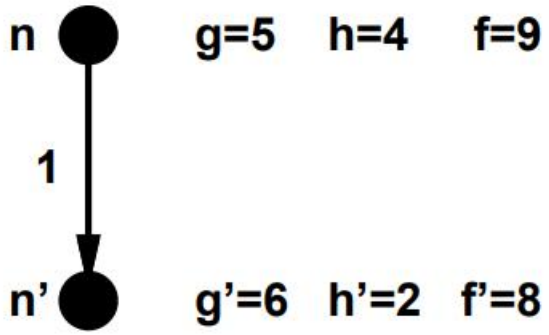
- $w'(u,v)$ is like an "amortized" cost of action $u \rightarrow v$. We require it to be always ≥ 0.
- We consider both the increase w of the current cost and the Δ on future costs through the change Δh of the potential function $h(.)$.

using the potential method

Pathmax

For some admissible heuristics, f may *decrease* along a path

E.g., suppose n' is a successor of n



But this throws away information!
$f(n) = 9 \Rightarrow$ true cost of a path through n is ≥ 9
Hence true cost of a path through n' is ≥ 9 also

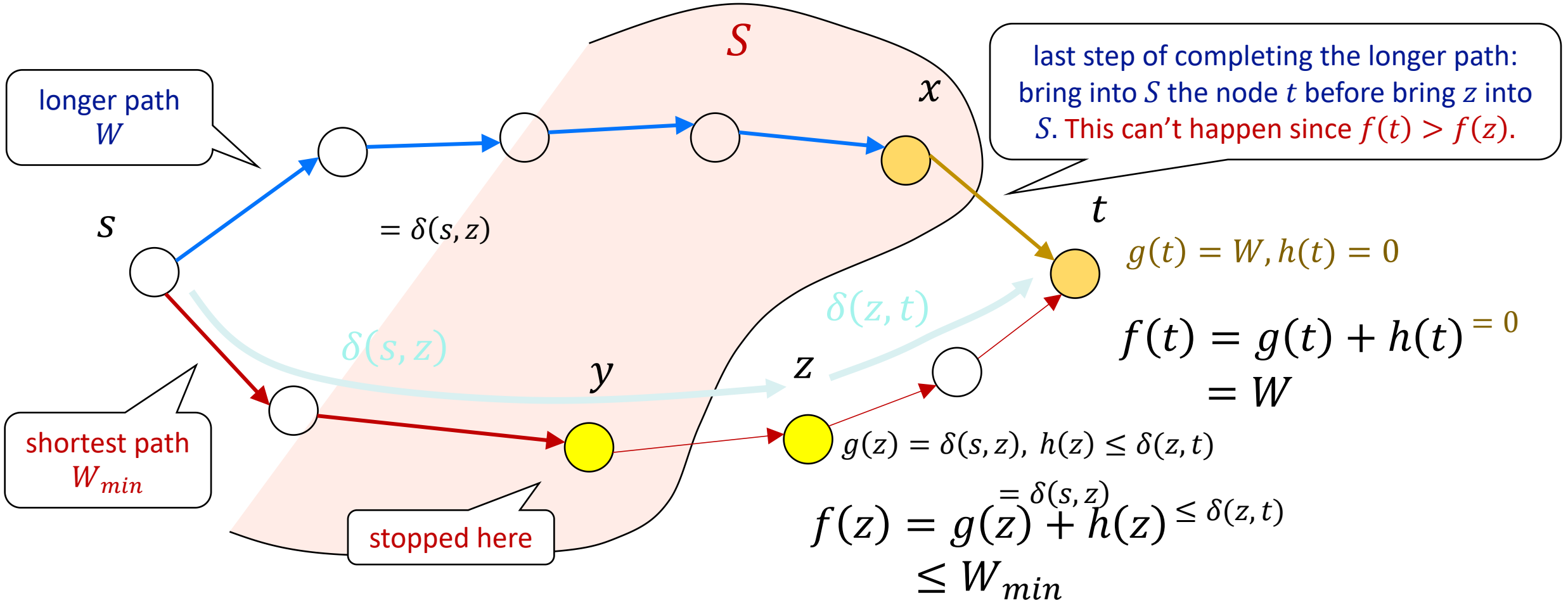
The resulting f is also always "consistent"

Pathmax modification to A*:
Instead of $f(n') = g(n') + h(n')$, use $f(n') = max(g(n') + h(n'), f(n))$

With pathmax, f is always nondecreasing along any path

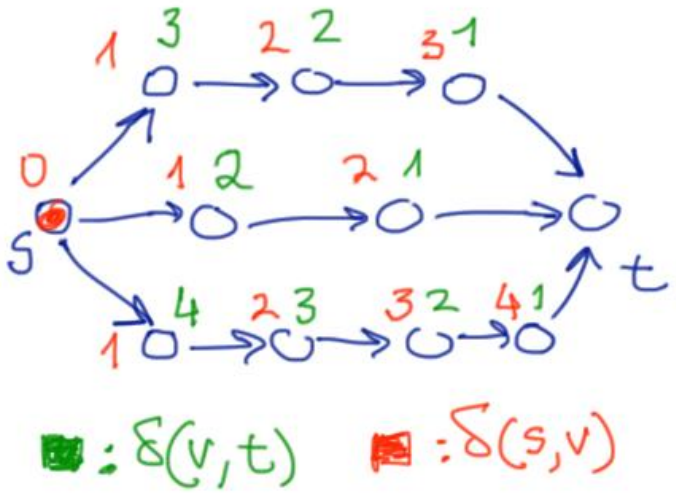
Admissible heuristic for destination t

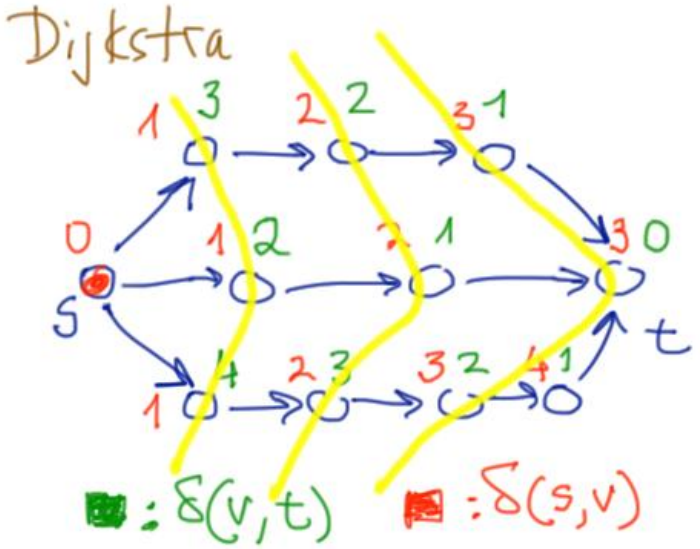
- Admissible $h(.)$: $h(z) \leq \delta(z,t)$.



Next

- Perform and explain A* simulations using https://qiao.github.io/PathFinding.js/visual/

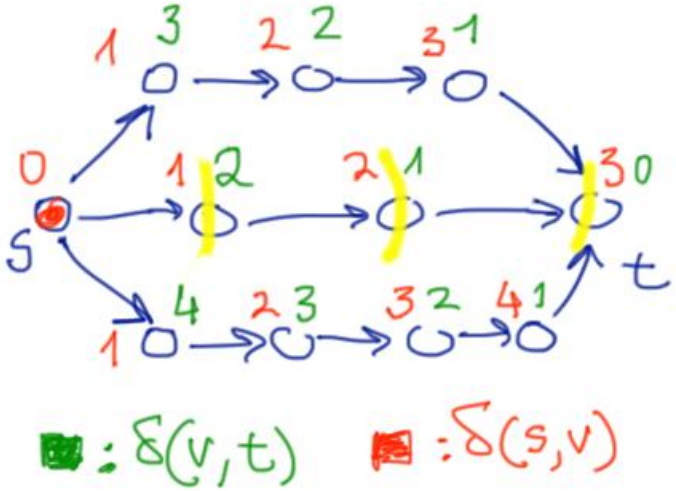




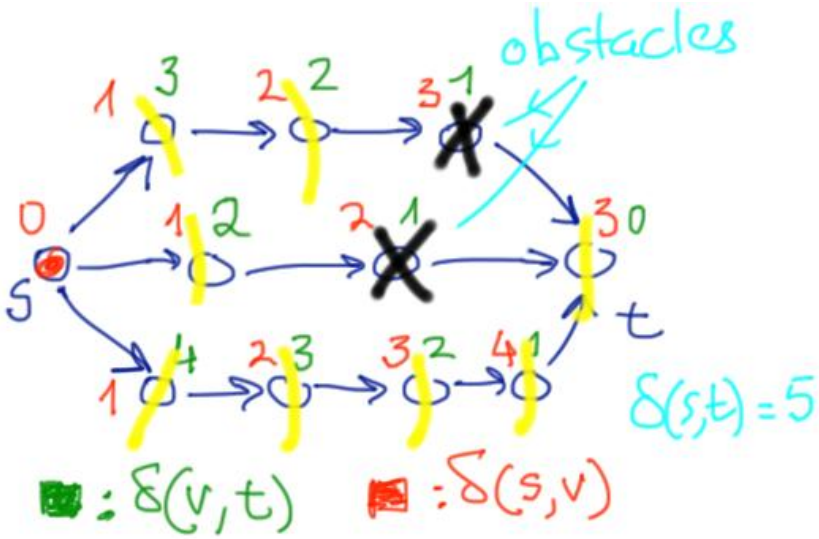
Note that in this example $h(v) = \delta(v,t)$ = actual shortest path distance.

Dijkstra explores nodes according to min $\delta(s,v)$

$h(v)$



A

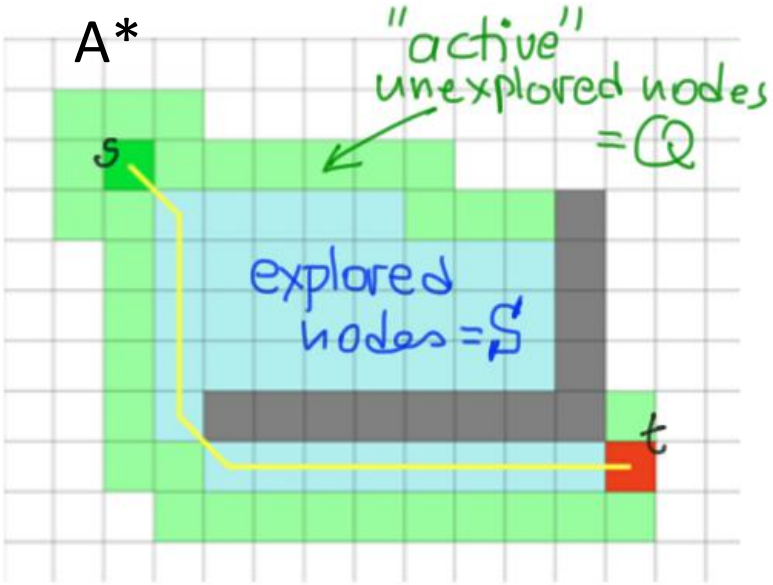


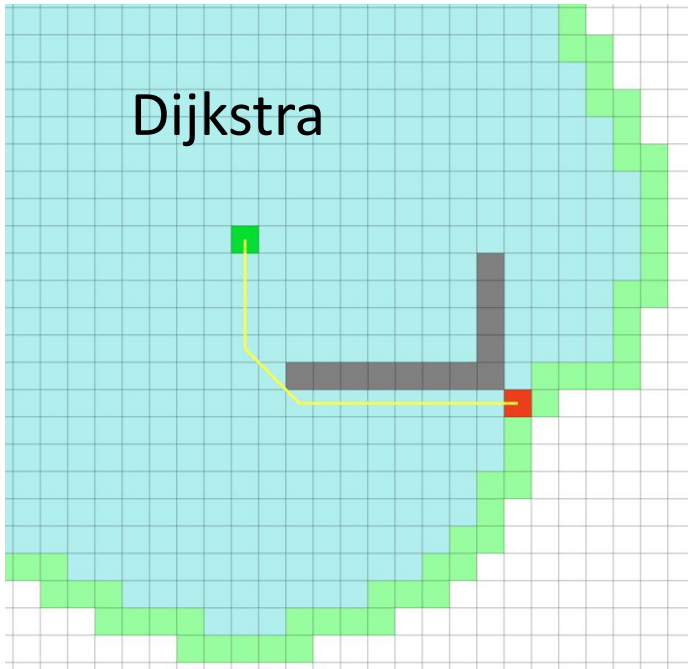
A* explores nodes according to min $\delta(s,v) + h(v) = \delta(s,v) + \delta(v,t)$

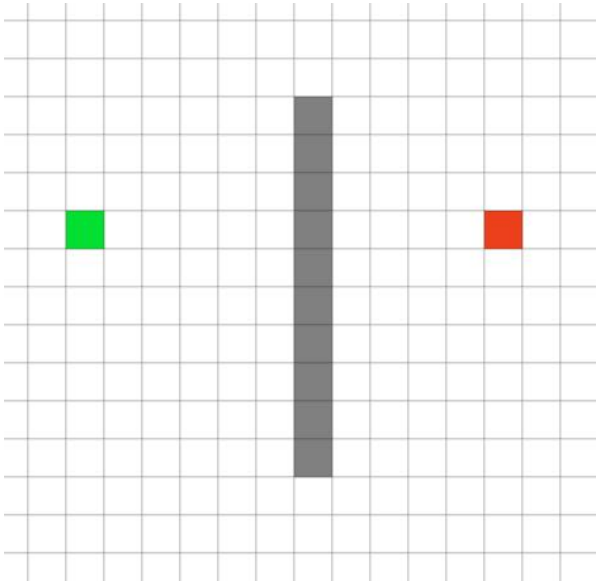
In general: $h(v) = \hat{\delta}(v,t)$ for some estimate $\hat{\delta}(v,t)$ of $\delta(v,t)$ where $\hat{\delta}(v,t) \leq \delta(v,t)$.

Note that if $\hat{\delta}(v,t) = \delta(v,t)$, A* explores the nodes along the shortest path.

Examples



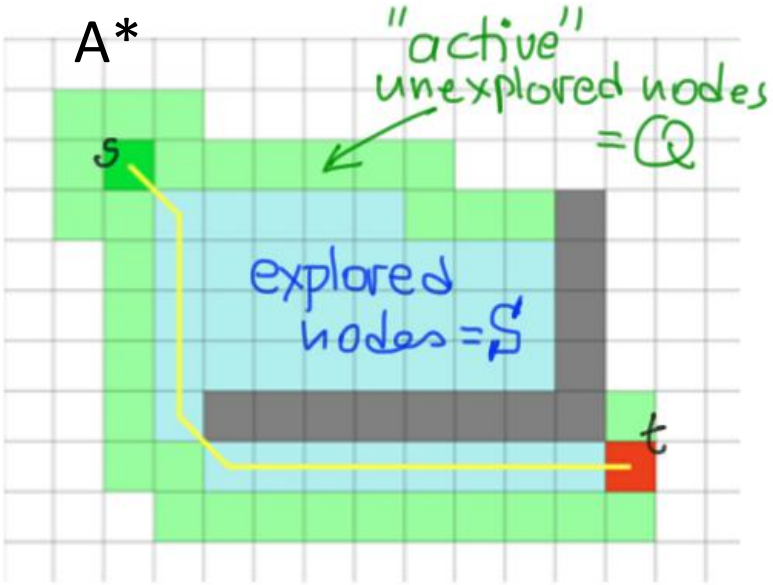


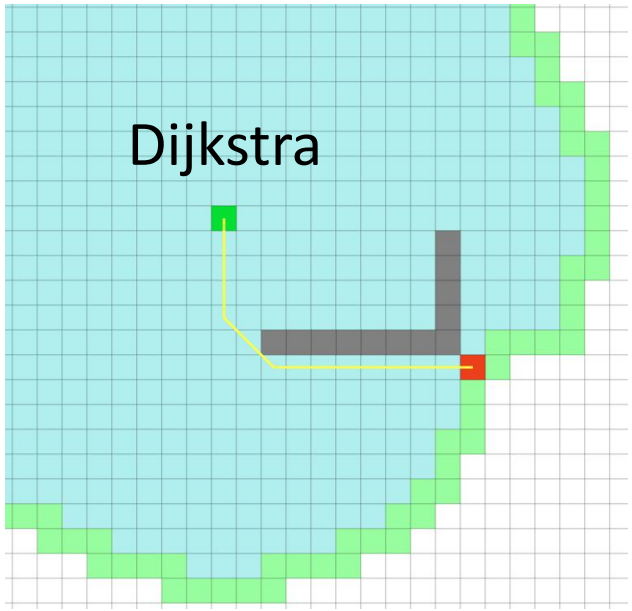


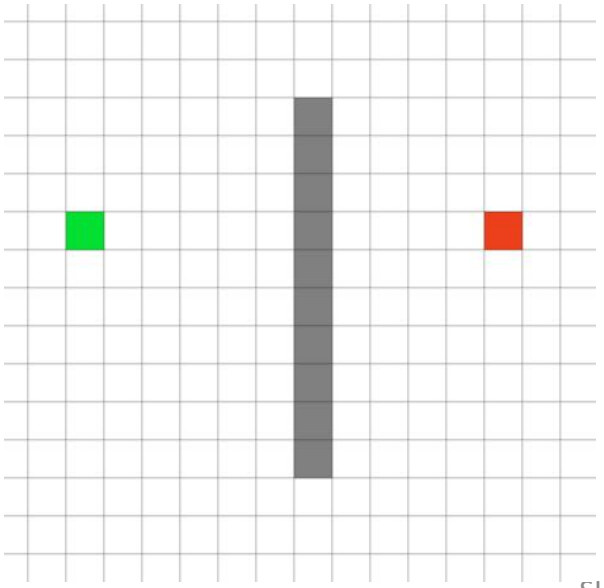
This link is running the code: try it yourself

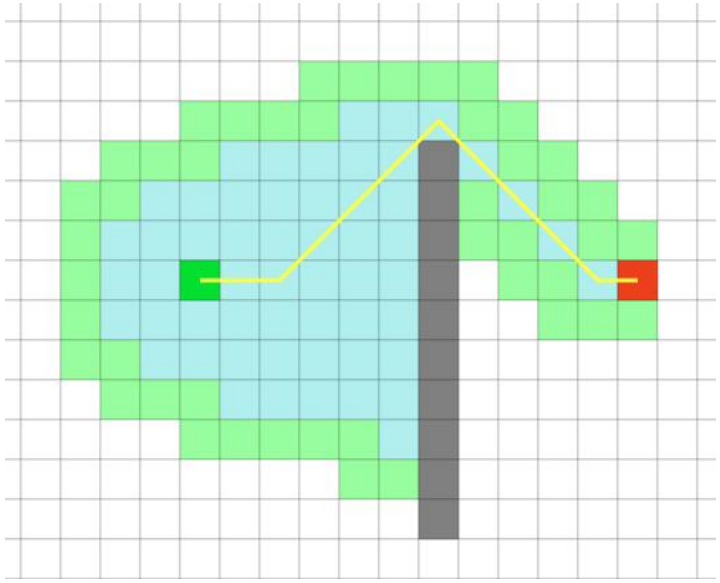
How do you expect the A* to work on this example?

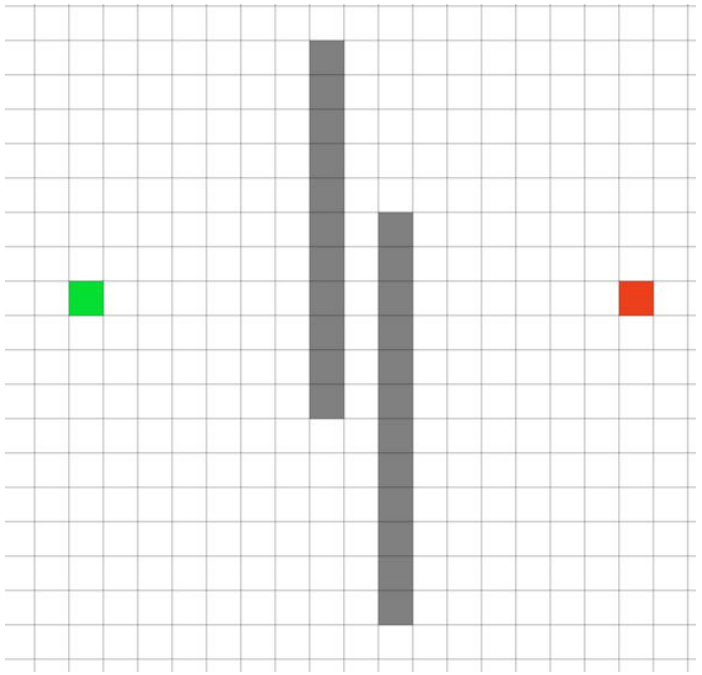
Examples

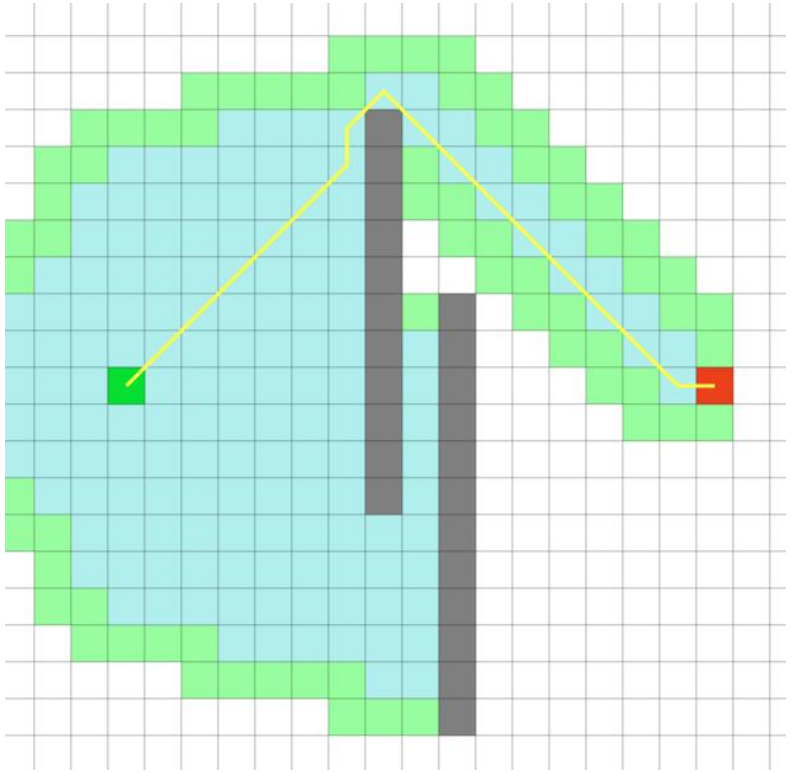












Comparing Dijkstra with A* video

Practical ways to define $h(.)$

- Euclidean distance for euclidean graphs. (Manhattan distance for grids)
- Use landmarks for general graphs.

  For each specific landmark node A, B, … pre-compute SP distances to and from all nodes of the graph.
- …
- Use solutions of relaxed problems, that are easy to solve.

Relaxed problems

Admissible heuristics can be derived from the *exact* solution cost of a *relaxed* version of the problem

If the rules of the 8-puzzle are relaxed so that a tile can move *anywhere*, then $h_1(n)$ gives the shortest solution

If the rules are relaxed so that a tile can move to *any adjacent square*, then $h_2(n)$ gives the shortest solution

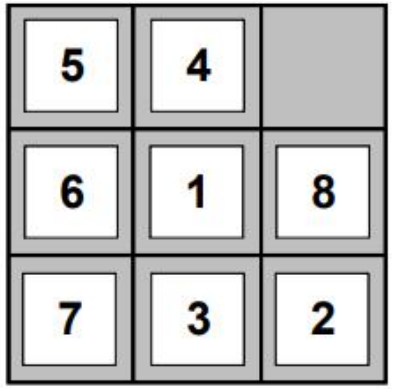
For TSP: let path be *any* structure that connects all cities

$\Longrightarrow$ minimum spanning tree heuristic

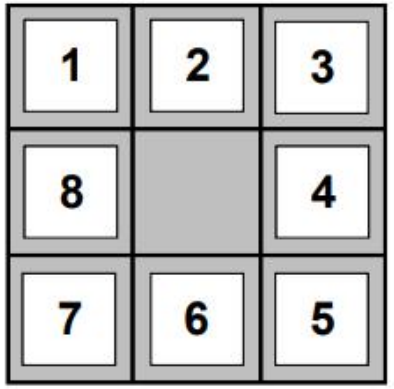
Admissible heuristics

E.g., for the 8-puzzle:

$h_1(n)$ = number of misplaced tiles
$h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)



Start State



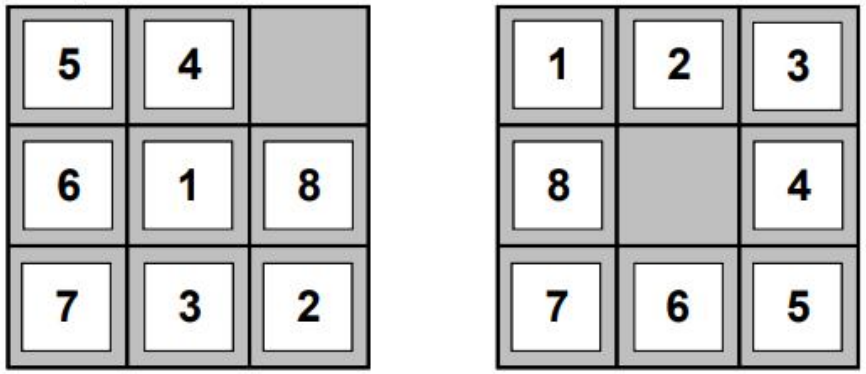
Goal State

$h_1(S)$ =??
$h_2(S)$ =??

Admissible heuristics

E.g., for the 8-puzzle:

$h_1(n)$ = number of misplaced tiles
$h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)



$h_1(S)$ =?? 7
$h_2(S)$ =?? 2+3+3+2+4+2+0+2 = 18

Dominance

If $h_2(n) \geq h_1(n)$ for all n (both admissible)
then h_2 *dominates* h_1 and is better for search

Typical search costs:

$d = 14$ IDS = 3,473,941 nodes
A*(h_1) = 539 nodes
A*(h_2) = 113 nodes
$d = 24$ IDS = too many nodes
A*(h_1) = 39,135 nodes
A*(h_2) = 1,641 nodes

The closer *h* is to the true cost to go, the better it is for search